SR UNIVERSITY

School of Computer Science and Artificial Intelligence

Lab Assignment # 6.5

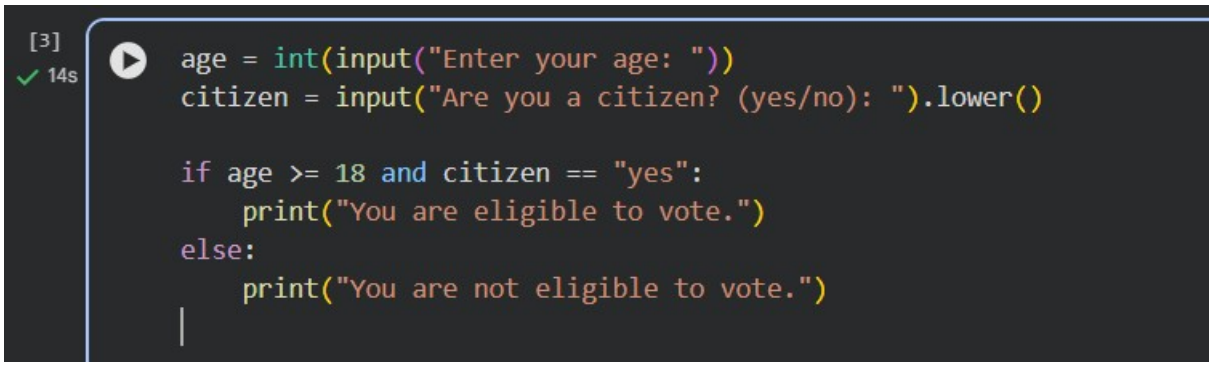
Program	**: B. Tech (CSE)**
Specialization	**:**
Course Title	**: AI Assisted coding**
Course Code	**:**
Semester	**: II**
Academic Session	**: 2025-2026**
Name of Student	**:B.Gnaneshwar**
Enrollment No.	**: 2403A51L15**
Batch No.	**: 51**
Date	**:20-01-2026**

TASK 1 :

Prompt :

"Generate Python code to check voting eligibility based on age and citizenship."

Code :



Explanation :

- `age` takes the user's age as input.
- `citizen` checks citizenship status.
- The `if` condition verifies:
- Age must be **18 or above**
- Citizenship must be **yes**

. If both conditions are satisfied, eligibility is confirmed.

Output :

```
Enter your age: 21
Are you a citizen? (yes/no): no
You are not eligible to vote.
```

TASK 2 :

Prompt :

Generate Python code to count vowels and consonants in a string using a loop.

Code :

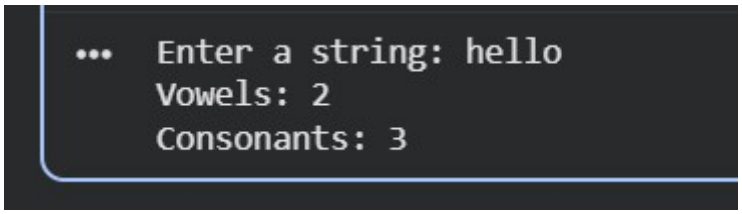
```
[4]
✓ 10s
text = input("Enter a string: ").lower()
vowels = "aeiou"
vowel_count = 0
consonant_count = 0

for char in text:
    if char.isalpha():
        if char in vowels:
            vowel_count += 1
        else:
            consonant_count += 1

print("Vowels:", vowel_count)
print("Consonants:", consonant_count)
```

Explanation :

- Converts input string to lowercase.
- Iterates through each character using a `for` loop.
- Checks if the character is an alphabet.

- Counts vowels and consonants separately.

Output :

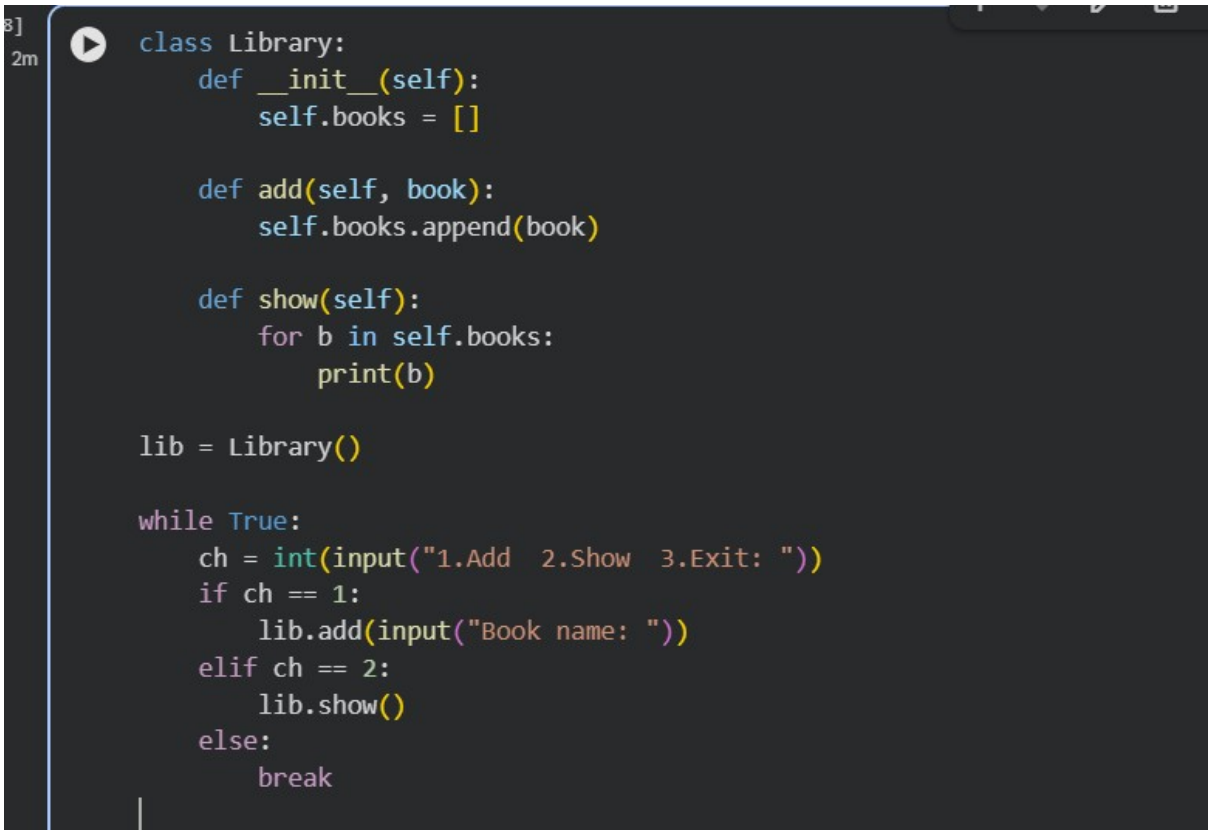


TASK 3 :

Prompt :

Generate a Python program for a library management system using classes, loops, and conditional statements.

Code :



Explanation :

- This task focused on developing a simple library management system using Python classes.

- It showcased object-oriented programming concepts with `Book` and `Library` classes,

- enabling operations like adding books and listing available inventory.

Output :

```
1.Add  2.Show  3.Exit: 1
Book name: python
1.Add  2.Show  3.Exit: 2
python
1.Add  2.Show  3.Exit: 3
```

TASK 4 :

Prompt :

Generate a Python class to mark and display student attendance using loops.

Code :

```python
class AttendanceSystem:
    def __init__(self):
        self.students = {}  # {student_id: name}
        self.attendance = {}  # {student_id: {date: status}}

    def add_student(self, student_id, name):
        if student_id not in self.students:
            self.students[student_id] = name
            self.attendance[student_id] = {}
            print(f"Added student: {name} (ID: {student_id})")
        else:
            print(f"Student with ID {student_id} already exists.")

    def mark_attendance(self, date_str, present_student_ids):
        print(f"\nMarking attendance for {date_str}:")
        for student_id, name in self.students.items():
            status = 'P' if student_id in present_student_ids else 'A'
            self.attendance[student_id][date_str] = status
            print(f"  {name} (ID: {student_id}): {status}")

    def display_attendance(self):
        print("\n--- Current Attendance Records ---")
        if not self.students:
            print("No students registered.")
            return
        for student_id, name in self.students.items():
            records = self.attendance.get(student_id, {})
            print(f"{name} (ID: {student_id}): {records}")
        print("----------------------------------")
```

Explanation :

- This task involved building an `AttendanceSystem` class to manage student attendance.

- It used dictionaries for data storage, loops for processing attendance records,

- and conditional logic for marking and displaying various attendance reports.

Output :

```
Added student: Alice (ID: S01)
Added student: Bob (ID: S02)

Marking attendance for 2023-11-01:
  Alice (ID: S01): P
  Bob (ID: S02): A

Marking attendance for 2023-11-02:
  Alice (ID: S01): P
  Bob (ID: S02): P

--- Current Attendance Records ---
Alice (ID: S01): {'2023-11-01': 'P', '2023-11-02': 'P'}
Bob (ID: S02): {'2023-11-01': 'A', '2023-11-02': 'P'}
---------------------------------
```

TASK 5 :

Prompt :

Generate a Python program using loops and conditionals to simulate an ATM menu.

Code :

```python
def atm_simulator_minimal():
    balance = 1000  # Initial balance

    print("Welcome to the Minimal ATM!")

    while True:
        print("\n--- Minimal ATM Menu ---")
        print("1. Check Balance")
        print("2. Exit")

        choice = input("Enter your choice (1-2): ")

        if choice == '1':
            print(f"Your current balance is: ${balance:.2f}")
        elif choice == '2':
            print("Thank you for using the Minimal ATM. Goodbye!")
            break
        else:
            print("Invalid choice. Please select 1 or 2.")

atm_simulator_minimal()
```

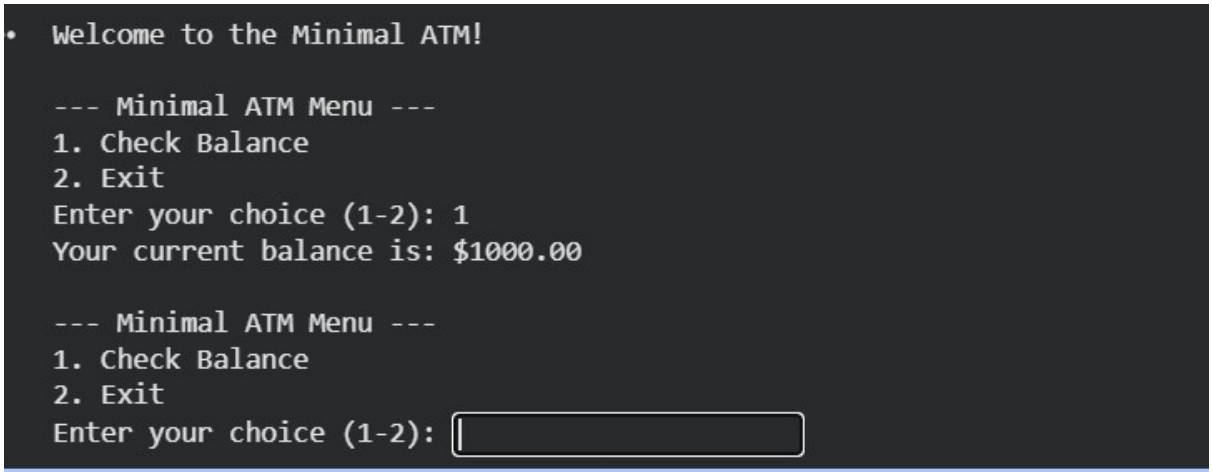
Explanation :

- This task focused on creating an interactive ATM simulator program.

- It employed `while` loops for continuous menu navigation and PIN verification,

- along with `if-elif-else` statements for handling user choices and transaction logic.

Output :



Final Conclusion :

- Throughout these five tasks, we explored fundamental Python programming concepts.

- We implemented conditional logic for voting eligibility, used loops for string processing

- (vowel/consonant counting), and applied object-oriented programming for a library
- and an attendance system. Finally, we created an interactive ATM simulator, demonstrating

- menu navigation with loops and conditional statements, thus covering a broad spectrum of programming constructs.